MongoDB & NoSQL Assignment Report

Installation · Data Modeling · CRUD Operations
Library Management System

Submitted by

Prince Jha

github.com/princejha-dev

Database Management Systems · 2026

Table of Contents

1.	Introduction to MongoDB & NoSQL	3
2.	Installing & Configuring MongoDB	3
3.	Understanding NoSQL Data Models	4
4.	Library System — Database Design	5
5.	CRUD Operations	6
6.	Search Queries & Advanced Operations	8
7.	Reflection — NoSQL in Modern Applications	10
8.	Conclusion	11

SECTION 1

1. Introduction to MongoDB & NoSQL

NoSQL (Not Only SQL) databases represent a departure from the traditional relational database model. Instead of organizing data in rigid tables with fixed schemas, NoSQL systems offer flexible, scalable data storage suited to modern application workloads — including real-time web apps, big data platforms, and distributed systems.

MongoDB is the world's most popular NoSQL database. It stores data as JSON-like documents (BSON — Binary JSON), making it a natural fit for JavaScript-based applications and APIs. Its document model mirrors how developers think about data, reducing the impedance mismatch between application code and the database layer.

Key Differences: SQL vs NoSQL

Feature	SQL (Relational)	NoSQL (MongoDB)
Data Model	Tables & Rows	Documents (BSON/JSON)
Schema	Fixed, predefined	Flexible, dynamic
Relationships	Foreign keys, JOINs	Embedded docs / references
Scalability	Vertical (scale up)	Horizontal (scale out)
Query Language	SQL	MongoDB Query Language (MQL)
Best For	ACID transactions	Large-scale, flexible data

SECTION 2

2. Installing & Configuring MongoDB

2.1 Installation on Ubuntu/Linux

Step 1 — Import the MongoDB public GPG key

```
wget -qO - https://www.mongodb.org/static/pgp/server-7.0.asc | sudo apt-key add -
```

Step 2 — Add the repository and install

```
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu
focal/mongodb-org/7.0 multiverse" \
| sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt-get update
sudo apt-get install -y mongodb-org
```

Step 3 — Start MongoDB

```
sudo systemctl start mongod
sudo systemctl enable mongod # start on boot
sudo systemctl status mongod # verify running
```

2.2 Installation on macOS (Homebrew)

```
brew tap mongodb/brew
brew install mongodb-community@7.0
brew services start mongodb-community@7.0
```

2.3 Configuration File

/etc/mongod.conf — key settings

```
storage:
dbPath: /var/lib/mongodb # data directory
net:
port: 27017 # default port
bindIp: 127.0.0.1 # localhost only (secure default)
security:
authorization: enabled # enable role-based access control
```

2.4 Connecting & Creating a User

```
mongosh # open the MongoDB shell
// Switch to admin database and create an admin user
use admin
db.createUser({
user: "princejha",
pwd: "securepassword",
roles: [{ role: "userAdminAnyDatabase", db: "admin" }]
})
```

SECTION 3

3. Understanding NoSQL Data Models

MongoDB organizes data in a three-tier hierarchy: Databases contain Collections, and Collections contain Documents. A document is a set of key-value pairs stored in BSON format.

3.1 Core Terminology

Term	SQL Equivalent	Description
Database	Database	Top-level container for collections
Collection	Table	Group of related documents
Document	Row	A single JSON-like data record
Field	Column	A key-value pair inside a document
_id	Primary Key	Auto-generated unique identifier (ObjectId)
Index	Index	Data structure to speed up queries
Aggregation	GROUP BY	Pipeline for data transformation & analysis

3.2 Embedding vs. Referencing

MongoDB offers two strategies for modeling relationships between data entities:

- **Embedding** — store related data inside a single document. Best for one-to-few relationships and data read together frequently.
- **Referencing** — store the ObjectId of another document. Best for one-to-many or many-to-many relationships where data changes independently.

Design Principle
In our Library System we use a hybrid approach: genres are embedded as arrays in book documents for fast read performance, while authors are referenced by ObjectId to avoid data duplication when an author writes multiple books.

SECTION 4

4. Library System — Database Design

The Library Management System uses a database named **libraryDB** with three collections: **books**, **authors**, and **genres**. Below are the document schemas and sample data.

4.1 Authors Collection

Sample document — authors

```
// authors collection
{
 id: ObjectId("64a1f2b3c4d5e6f7a8b9c0d1"),
name: "George Orwell",
nationality: "British",
birthYear: 1903,
deathYear: 1950,
bio: "English novelist, essayist, and critic.",
books: [], // populated via reference from books collection
createdAt: ISODate("2024-01-15T10:00:00Z")
}
```

4.2 Genres Collection

Sample document — genres

```
// genres collection
{
 id: ObjectId("64a1f2b3c4d5e6f7a8b9c0d4"),
name: "Dystopian Fiction",
description: "Fiction set in a repressive or nightmarish society.",
popularity: "High"
}
```

4.3 Books Collection

Sample document — books (central collection)

```
// books collection
{
 id: ObjectId("64a1f2b3c4d5e6f7a8b9c0d7"),
title: "Nineteen Eighty-Four",
authorId: ObjectId("64a1f2b3c4d5e6f7a8b9c0d1"), // ref to authors
genres: ["Dystopian Fiction", "Political Fiction", "Science Fiction"],
isbn: "978-0-452-28423-4",
publishYear: 1949,
pages: 328,
language: "English",
rating: 4.8,
available: true,
copies: 5,
createdAt: ISODate("2024-01-15T10:00:00Z")
}
```

4.4 Inserting Sample Data

Set up the database and insert records

```
use libraryDB
// Insert authors
db.authors.insertMany([
{ name: "George Orwell", nationality: "British", birthYear: 1903 },
{ name: "J.K. Rowling", nationality: "British", birthYear: 1965 },
{ name: "Gabriel Garcia Marquez", nationality: "Colombian", birthYear: 1927 },
{ name: "Toni Morrison", nationality: "American", birthYear: 1931 }
])
// Insert genres
db.genres.insertMany([
{ name: "Dystopian Fiction", popularity: "High" },
{ name: "Fantasy", popularity: "High" },
{ name: "Magical Realism", popularity: "Medium" },
{ name: "Literary Fiction", popularity: "High" }
])
// Insert books
db.books.insertMany([
{ title: "Nineteen Eighty-Four", authorId: ObjectId("..."), genres: ["Dystopian
Fiction"], rating: 4.8, publishYear: 1949, available: true, copies: 5 },
{ title: "Animal Farm", authorId: ObjectId("..."), genres: ["Political Fiction"],
rating: 4.5, publishYear: 1945, available: true, copies: 3 },
{ title: "Harry Potter and the Sorcerer's Stone", authorId: ObjectId("..."),
genres: ["Fantasy"], rating: 4.9, publishYear: 1997, available: true, copies: 8 },
{ title: "One Hundred Years of Solitude", authorId: ObjectId("..."), genres:
["Magical Realism"], rating: 4.7, publishYear: 1967, available: false, copies: 0 },
{ title: "Beloved", authorId: ObjectId("..."), genres: ["Literary Fiction"],
rating: 4.6, publishYear: 1987, available: true, copies: 2 }
])
```

SECTION 5

5. CRUD Operations

CRUD stands for Create, Read, Update, and Delete — the four fundamental operations of any database system. Below are practical examples applied to our Library System.

5.1 Create — insertOne() & insertMany()

```
// INSERT a single new book
db.books.insertOne({
title: "The Alchemist",
authorId: ObjectId("64a1..."),
genres: ["Adventure", "Philosophical Fiction"],
isbn: "978-0-06-112241-5",
publishYear: 1988,
pages: 197,
rating: 4.7,
available: true,
copies: 6
})
// INSERT multiple authors at once
db.authors.insertMany([
{ name: "Paulo Coelho", nationality: "Brazilian", birthYear: 1947 },
{ name: "Haruki Murakami",nationality: "Japanese", birthYear: 1949 }
])
```

5.2 Read — find() & findOne()

```
// Find ALL books
db.books.find({})
// Find ONE specific book by title
db.books.findOne({ title: "Animal Farm" })
// Find books with rating above 4.5 — show title and rating only
db.books.find({ rating: { $gt: 4.5 } }, { title: 1, rating: 1, _id: 0 })
// Find all available books, sorted by rating descending
db.books.find({ available: true }).sort({ rating: -1 })
// Find books published before 1960
db.books.find({ publishYear: { $lt: 1960 } })
```

5.3 Update — updateOne() & updateMany()

```
// UPDATE the number of copies for one book
db.books.updateOne(
{ title: "Beloved" },
{ $set: { copies: 4, available: true } }
)
// ADD a genre tag to a book using $push
db.books.updateOne(
{ title: "The Alchemist" },
{ $push: { genres: "Self-Help" } }
)
// Increment copies by 2 for all available books
db.books.updateMany(
{ available: true },
{ $inc: { copies: 2 } }
)
// UPDATE author bio
db.authors.updateOne(
{ name: "Toni Morrison" },
{ $set: { bio: "Nobel Prize-winning American novelist (1993)." } }
)
```

5.4 Delete — deleteOne() & deleteMany()

```
// DELETE a single book by title
db.books.deleteOne({ title: "Animal Farm" })
// DELETE all books with 0 copies
db.books.deleteMany({ copies: { $eq: 0 } })
// DELETE an author (use with caution — check references first)
db.authors.deleteOne({ name: "Paulo Coelho" })
```

SECTION 6

6. Search Queries & Advanced Operations

6.1 Searching Books by Various Criteria

Text search, genre filter, and range queries

```
// Create a text index for full-text search
db.books.createIndex({ title: "text" })

// Full-text search by keyword
db.books.find({ $text: { $search: "solitude" } })

// Search books by genre (array contains value)
db.books.find({ genres: "Fantasy" })

// Books within a rating range
db.books.find({ rating: { $gte: 4.5, $lte: 5.0 } })

// Books published between 1940 and 1970
db.books.find({
publishYear: { $gte: 1940, $lte: 1970 }
})

// Available books with more than 3 copies
db.books.find({ available: true, copies: { $gt: 3 } })

// Books NOT in a language (using $ne)
db.books.find({ language: { $ne: "French" } })
```

6.2 Searching Authors

```
// Find British authors
db.authors.find({ nationality: "British" })

// Authors born before 1950
db.authors.find({ birthYear: { $lt: 1950 } })

// Search by name using regex (case-insensitive)
db.authors.find({ name: { $regex: /morrison/i } })

// Authors from multiple nationalities using $in
db.authors.find({
nationality: { $in: ["American", "Colombian"] }
})
```

6.3 JOIN with $lookup (Aggregation)

Since books reference authors by ObjectId, we use $lookup to join the collections — similar to SQL JOINs.

```
// Join books with their author details
db.books.aggregate([
{
$lookup: {
from: "authors",
localField: "authorId",
foreignField: " id",
as: "authorDetails"
}
},
{ $unwind: '$authorDetails' },
{
$project: {
title: 1,
rating: 1,
"authorDetails.name": 1,
"authorDetails.nationality": 1
}
}
])
```

6.4 Aggregation Pipeline

Analytics — average rating per genre

```
db.books.aggregate([
// Step 1: Flatten the genres array
{ $unwind: '$genres' },
// Step 2: Group by genre and compute stats
{
$group: {
 id: "$genres",
avgRating: { $avg: "$rating" },
totalBooks:{ $sum: 1 },
maxRating: { $max: "$rating" }
}
},
// Step 3: Sort by average rating descending
{ $sort: { avgRating: -1 } },
// Step 4: Limit to top 5 genres
{ $limit: 5 }
])
```

6.5 Indexing for Performance

```
// Single-field index
db.books.createIndex({ rating: -1 })

// Compound index for common query pattern
db.books.createIndex({ available: 1, rating: -1 })

// Unique index on ISBN
db.books.createIndex({ isbn: 1 }, { unique: true })

// List all indexes on the books collection
db.books.getIndexes()

// Analyze query performance
db.books.find({ available: true }).explain("executionStats")
```

SECTION 7

7. Reflection — NoSQL in Modern Applications

7.1 Why NoSQL Has Become Essential

The rise of the internet, mobile applications, and big data has changed what we expect from databases. Applications now handle massive volumes of data from diverse sources, require near-instant response times, and must scale across distributed global infrastructure. Traditional relational databases, while powerful, were designed for a different era — one where data was structured, predictable, and fit neatly into tables.

NoSQL systems emerged to address these challenges. They prioritize horizontal scalability (adding more servers rather than upgrading one), schema flexibility (data structures can evolve without migrations), and high availability (data is replicated across multiple nodes automatically).

7.2 Real-World Use Cases

E-Commerce	Product catalogs with varying attributes (clothing has size/color, electronics have specs)
Social Media	User profiles, posts, and activity feeds with high write throughput
IoT Platforms	Sensor time-series data arriving at millions of events per second
Content Management	Articles, media, and metadata with evolving schema requirements
Real-Time Analytics	Aggregating user behavior data for dashboards and recommendations
Gaming	Player state, leaderboards, and session data requiring low-latency reads/writes

7.3 Advantages of MongoDB Observed in This Assignment

- **Schema flexibility:** Adding new fields to book documents (e.g., 'language', 'isbn') required no migration — simply insert documents with the new fields.
- **Natural data representation:** The genres array inside book documents is intuitive — no junction table needed as in SQL.
- **Powerful aggregation:** The aggregation pipeline allowed complex analytics (average rating by genre, author book counts) in a single, readable query chain.
- **Fast development cycle:** Getting from zero to a functional database with sample data and queries took a fraction of the time of a relational setup.

- **Rich query operators:** Operators like $regex, $in, $lookup, and $text search made it easy to implement sophisticated search functionality.

7.4 Limitations & Considerations

- **No joins natively:** Multi-collection queries require the $lookup aggregation stage, which can be verbose and slower than native SQL JOINs for complex relationships.
- **ACID transactions:** While MongoDB supports multi-document transactions since v4.0, they are more complex to implement than in traditional RDBMS.
- **Data duplication risk:** Embedding data for performance can lead to inconsistencies if the same data appears in multiple documents.
- **Memory usage:** BSON format stores field names in every document, which can increase storage overhead for collections with many fields.

Personal Reflection

Working with MongoDB on this Library System highlighted how document-oriented thinking differs from relational thinking. The hardest shift was deciding when to embed vs. reference — a decision that has major performance implications. The flexibility is powerful, but it demands disciplined schema design to avoid chaotic, inconsistent collections. Overall, I found MongoDB's developer experience significantly faster for prototyping, and its aggregation pipeline surprisingly expressive for analytics.

SECTION 8

8. Conclusion

This assignment provided hands-on experience with MongoDB across its key areas: installation and configuration, data modeling, CRUD operations, search queries, and aggregation pipelines. Through the Library Management System, I implemented a practical database that demonstrates the strengths of the document model for real-world applications.

The core takeaway is that NoSQL is not a replacement for relational databases, but a complementary tool. MongoDB excels when data is hierarchical, schema varies between records, read/write throughput is paramount, or horizontal scaling is required. For applications with complex transactional requirements and highly normalized data, relational systems remain the right choice.

As modern applications grow more diverse and data-intensive, understanding both SQL and NoSQL paradigms is an essential skill for any developer or database administrator. MongoDB's continued adoption by companies like Forbes, Toyota, eBay, and Coinbase confirms its place as a cornerstone of the modern data stack.

Assignment Summary

Topic	Status	Details
Installation & Setup	**Complete**	Ubuntu + macOS instructions, config file, user creation
Data Modeling	**Complete**	3 collections: books, authors, genres; embed vs referenc
CRUD Operations	**Complete**	insertOne/Many, find/findOne, update, delete with operato
Search Queries	**Complete**	Text search, regex, range, $in, multi-criteria filters
Advanced Operations	**Complete**	$lookup join, aggregation pipeline, indexing, explain()
Reflection	**Complete**	Use cases, advantages, limitations, personal insights

e

Prince Jha
github.com/princejha-dev
Database Management Systems — MongoDB Assignment — 2026